

Lab 09-2: ECO-Integer Division (2%)

Assigned on Oct 30, 2025

Due day on **Nov 6, 2025**

Objective

In this lab, you will learn:

1. How to do ECO to correct the bugs in synthesized netlist.

Demo checklist:

- ☐ (Mission 1) The modified *inverse_table.v*. [Only two lines should be changed compared to the original one, and it should pass the simulation using *test_my_div.f* but fail using *test_my_div_buggy.f*]
- ☐ (Mission 2) Synthesized netlist *my_div_syn.v*, its area and timing report, and the log file of LEC for verifying the equivalence between the design in *test_my_div.f* and this netlist.
- ☐ (Mission 3) The corrected RTL *inverse_table_corrected.v*.
- ☐ (Mission 4) The ECO-modified netlist *my_div_syn_corrected.v* and the log of LEC for verifying the equivalence between the design in *test_my_div_corrected.f* and this netlist.

Environment Setup

Copy lab file packages from ee4292. Decompress the package and enter it. You can check the file list in Appendix.

```
$ cp ~ee4292/iclab2025/lab09.zip .
```

```
$ unzip lab09.zip
```

```
$ cd lab09_part2/
```

Note: please run simulation in the *sim* directory to maintain a fine data management.

Description

You are a digital designer for a world-wide famous IC design house: NTHUTek. One day, when you walked in the company, you were told that one of your colleagues was fired because his design in a flagship project completely failed. Your boss asked you to take charge of this design. So, try your best to fix the problem (and avoid getting fired)! Good luck!

The function of this design, *my_div*, is non-negative integer division as shown in Fig.

1. For 16-bit input *dividend* and 5-bit input *divisor*, this block generates a 16-bit integer

quotient by converting the division into multiplication by

$$\frac{\text{dividend}}{\text{divisor}} = \text{dividend} \times \frac{2^{\text{div_shift}}}{\text{divisor}} \times \frac{1}{2^{\text{div_shift}}} = (\text{dividend} \times \text{div_inverse}) \gg \text{div_shift}.$$

The module *inverse_table* is the look-up table for *div_inverse* and *div_shift*, and the *mul_and_shift* is responsible for the multiplication and shifting.

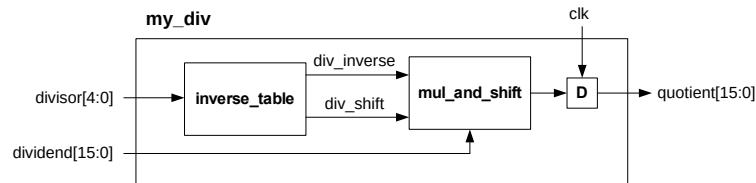


Fig. 1. Block diagram of *my_div*

Action Items

Mission 1:

After reading the RTL codes of your lazy and careless ex-colleague, you found that he didn't prepare a solid testbench to test this module and this is why the design completely fails. So, you wrote a testbench Verilog code *test_my_div.v* and found there were two bugs in *inverse_table.v*. Please fix them by modifying the RTL code *inverse_table.v* directly.

Mission 2:

You need to handover your netlist, *my_div_syn.v*, to the backend team. Please run logic synthesis using the synthesis script prepared by the backend team, and also run LEC (copy LEC scripts from lab09_part1 and modify them) to verify the logic equivalence. Then you can enjoy your long vacation!

```
$ bash run_dc.bat
```

Mission 3:

Today, three months later since Mission 2, it suddenly occurred to you that your testbench DOES NOT cover all possible inputs. The 16-bit *dividend* should range from 0 to 65535, instead of only 0 to 32767 in *test_my_div.v*. You then quickly modified a new testbench (*test_my_div_corrected.v*), ran the simulation, and prayed. Now, the infamous story of “Pentium FDIV bug” starts replaying in your mind because you got hidden bugs in YOUR *inverse_table.v*. Fix the bugs and save the code to *inverse_table_corrected.v*.

Mission 4:

Unfortunately, the backend team doesn't allow you to re-synthesize your RTL code. Instead, they asked you to do ECO using only three kinds of available spare cells: NAND, NOR, and INV. You have no choice, so please do ECO and use LEC to verify the logic equivalence between the correct RTL codes and the modified netlist *my_div_syn_corrected.v*.

****Only three standard cells are allowed:**

INVERTER: INVXL (A, Y); //Y=~A

NAND: NAND2XL (A, B, Y); //Y=~(A1&A2)

NOR: NOR2XL (A, B, Y); //Y=~(A1|A2)

Appendix

Directory	Description
<i>source/</i>	Your ex-colleague's RTL codes: <i>my_div.v</i> , <i>inverse_table.v</i> , and <i>mul_and_shift.v</i> . Put your modified <i>inverse_table.v</i> and <i>inverse_table_corrected.v</i> here for running simulation.
<i>sim/</i>	Testbench and file lists (Don't modify them): <i>test_my_div.f</i> for Mission 1; <i>test_my_div_buggy.f</i> for Mission 3 (buggy RTL); <i>test_my_div_corrected.f</i> for Mission 3 (corrected RTL).
<i>syn/</i>	Synthesis environment prepared by the backend team. (Don't modify them) Run synthesis using the batch file <i>./run_dc.bat</i> .
<i>netlist/</i>	Synthesized gate-level netlists (copied by <i>syn/run_dc.bat</i> automatically): <i>my_div_syn.v</i> Put your modified <i>my_div_syn_corrected.v</i> here and run LEC in <i>lec/</i> to verify the logic equivalence (Do not forget to modify LEC scripts!)
<i>lec/</i>	Please copy LEC scripts from lab09_part1 and modify reference scripts by yourself